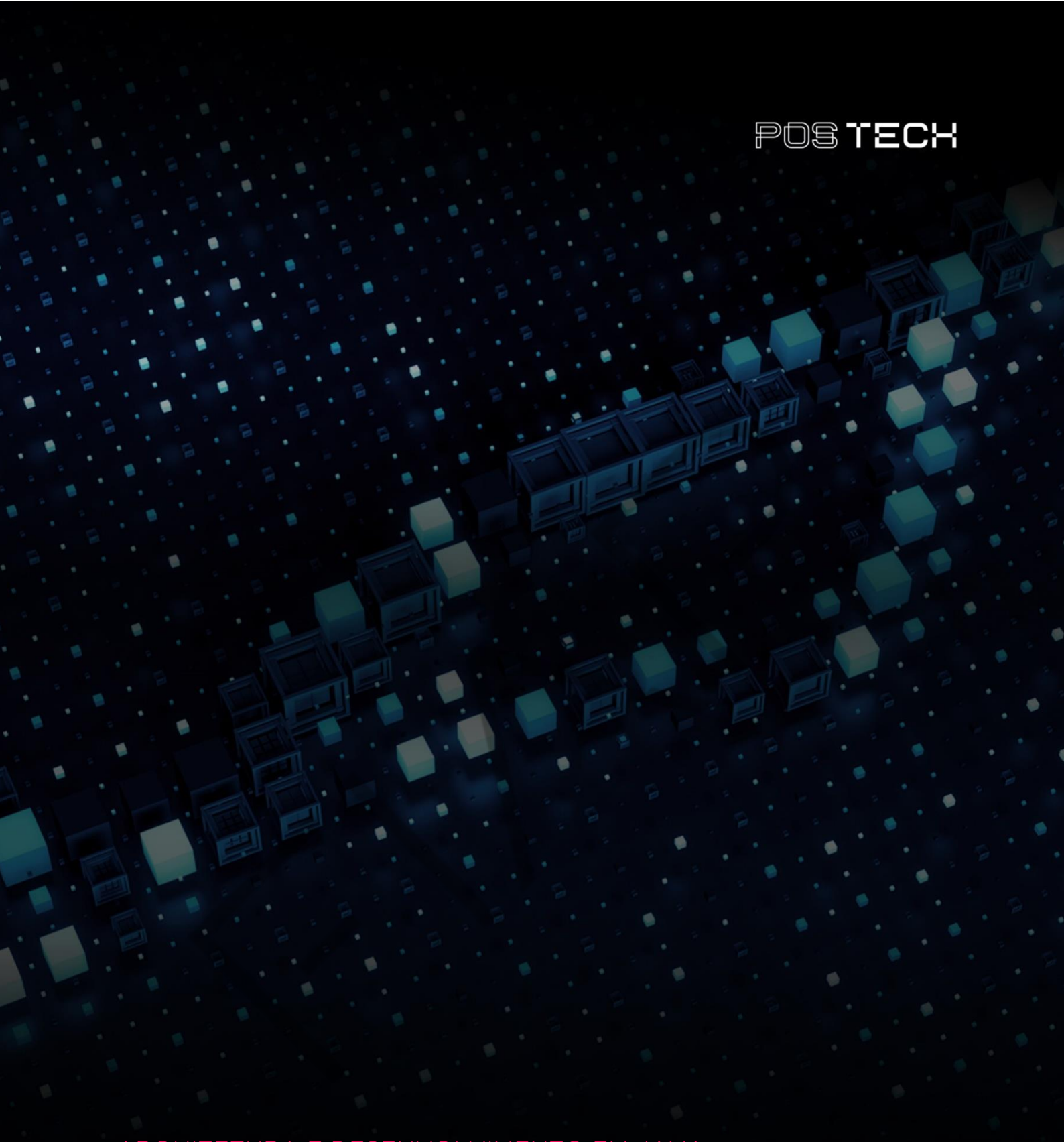


POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 04 -

INTRODUÇÃO AO RABBITMQ

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON.....4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA?.....9

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Exploraremos os fundamentos do RabbitMQ, um broker de mensagens amplamente utilizado para integrar sistemas e processar mensagens de forma assíncrona. Começaremos com uma introdução ao RabbitMQ, destacando sua definição e aplicações práticas em cenários reais.

Discutiremos o funcionamento básico da arquitetura do RabbitMQ, explicando como mensagens são enviadas pelos producers para exchanges, roteadas para queues com base em regras e finalmente consumidas pelos consumers. Destacaremos a abordagem assíncrona do RabbitMQ, que permite o processamento independente entre produtores e consumidores.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Abordaremos os passos teóricos necessários para configurar e trabalhar com o RabbitMQ em sistemas distribuídos. Começaremos com uma visão geral da configuração básica do RabbitMQ, explicando como criar exchanges, filas e estabelecer relações entre elas por meio de bindings.

Discutiremos os diferentes tipos de exchanges, como direct, fanout e topic, detalhando como elas roteiam mensagens para filas com base em routingkeys e outras regras de roteamento. Também exploraremos como os producers enviam mensagens para exchanges e como os consumers consomem mensagens das filas, destacando a independência entre os processos de produção e consumo.

Além disso, abordaremos conceitos avançados, como configuração de filas duráveis para garantir persistência de mensagens e o uso de confirmações (ack) para assegurar que mensagens sejam processadas corretamente.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

RabbitMQ é um broker de mensagens assíncrono utilizado para facilitar a comunicação entre sistemas, atuando como um intermediário no processamento de mensagens entre produtores (producers) e consumidores (consumers). Ele organiza as mensagens em filas, permitindo que diferentes partes de uma aplicação ou sistemas separados troquem informações de forma eficiente, mesmo quando não estão disponíveis ao mesmo tempo.

Uma das principais aplicações práticas do RabbitMQ é no processamento de pedidos em e-commerce. Ele também é amplamente usado na integração de sistemas em setores diversos, como logística, transporte e faturamento. Ele permite que sistemas heterogêneos compartilhem dados de forma confiável e organizada, mesmo em cenários de alta complexidade.

Imagine um cenário de compra on-line. Quando o pedido é realizado, uma mensagem é enviada para uma fila de distribuição, onde o serviço responsável pelo estoque a consome e confirma a disponibilidade do produto. Outra mensagem pode ser enviada para a fila de transporte, onde o sistema logístico organiza a entrega. Simultaneamente, uma mensagem é processada na fila de faturamento, gerando a nota fiscal. Essa organização eficiente e assíncrona evita atrasos e promove maior flexibilidade na comunicação entre sistemas. A separação em filas garante que cada serviço funcione de maneira independente e escalável.

Com sua capacidade de lidar com diferentes padrões de mensageria e alta confiabilidade, o RabbitMQ é uma escolha a ser considerada para arquiteturas que dependem de troca eficiente e desacoplada de informações entre sistemas.O funcionamento do RabbitMQ se baseia em uma arquitetura flexível que garante a comunicação eficiente entre produtores e consumidores por meio de mensagens.

Na arquitetura geral, os produtores enviam mensagens para componentes chamados exchanges, que atuam como roteadores. As exchanges são configuradas com regras específicas que determinam para quais filas as mensagens devem ser enviadas. Cada fila, por sua vez, é consumida por um ou mais consumidores, que leem e processam as mensagens de acordo com sua lógica de negócio.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O RabbitMQ utiliza um padrão assíncrono, permitindo que as mensagens sejam processadas de maneira independente do tempo de execução do consumidor. Isso significa que os produtores podem publicar mensagens sem precisar esperar que os consumidores as processem imediatamente. Essa abordagem desacopla os sistemas, garantindo que o fluxo de dados continue fluindo mesmo quando os consumidores estão temporariamente indisponíveis ou lidando com altas cargas de trabalho.

Esse modelo de funcionamento é ideal para sistemas distribuídos, permitindo escalabilidade, flexibilidade e resiliência no processamento de mensagens. Com a capacidade de organizar e priorizar o tráfego de mensagens, o RabbitMQ se torna uma solução assertiva para implementar arquiteturas assíncronas em diversos tipos de aplicações.

No contexto do RabbitMQ, um Producer é o sistema ou aplicação responsável por criar e enviar mensagens para o broker. Ele atua como o ponto inicial no fluxo de mensageria, gerando informações que precisam ser processadas ou compartilhadas por outros sistemas ou serviços.

Os producers não interagem diretamente com as filas onde as mensagens são armazenadas. Em vez disso, eles enviam mensagens para as exchanges, que têm a função de roteá-las para as filas apropriadas, com base nas regras de roteamento configuradas. Essa separação entre produtores, exchanges e filas garante flexibilidade e desacoplamento no sistema.

No nosso exemplo de e-commerce, o sistema responsável por registrar novos pedidos pode atuar como um producer, enviando mensagens para uma exchange. Essas mensagens podem conter informações como o ID do pedido, os itens comprados e os dados do cliente. A exchange, então, roteia essas mensagens para filas específicas, onde serão consumidas por serviços como controle de estoque, faturamento e logística.

Assim como no Kafka, o papel do producer é fundamental, pois ele inicia o fluxo de comunicação, garantindo que as mensagens sejam entregues ao RabbitMQ para posterior processamento pelos consumidores. Essa estrutura permite que os producers se concentrem apenas na criação e envio das mensagens, enquanto o RabbitMQ gerencia todo o fluxo de mensageria.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O direcionamento de mensagens por uma Exchange é essencial para organizar o fluxo de mensagens em sistemas distribuídos, permitindo que diferentes tipos de dados ou eventos sejam direcionados para os serviços apropriados. Por exemplo, em uma aplicação de logística, uma exchange pode receber mensagens de um producer que reporta eventos de envio de pacotes e atualizações de status. Com base nas regras configuradas, a exchange pode roteá-las para filas distintas: uma fila para o serviço de transporte e outra para o sistema de rastreamento.

Assim como no Kafka, no RabbitMQ temos o conceito de filas (Queue). Uma Queue é o local onde as mensagens são armazenadas temporariamente até que sejam consumidas. Ela funciona como um buffer que garante que os dados enviados pelos producers fiquem disponíveis para os consumidores, mesmo que eles não estejam prontos para processá-los imediatamente.

As filas são organizadas de forma sequencial, garantindo que as mensagens sejam entregues na ordem em que foram recebidas. Além disso, elas podem ser configuradas com diferentes propriedades, como durabilidade, para assegurar que as mensagens permaneçam armazenadas mesmo em caso de reinicialização do sistema.

Trazendo novamente para o cenário de e-commerce, mensagens relacionadas a novos pedidos podem ser armazenadas em uma fila específica até que sejam consumidas por serviços de controle de estoque ou processamento de pagamento. Essa estrutura permite que cada consumidor processe as mensagens no seu próprio ritmo, sem depender diretamente do tempo de execução dos producers.

Se temos queue e producers, temos que ter também os consumers. O consumer é o sistema ou aplicação responsável por consumir mensagens das filas. Ele atua como o destinatário das mensagens enviadas pelos producers e armazenadas pelo broker, processando os dados conforme a lógica de negócio necessária.

Os consumidores se conectam a filas específicas e leem as mensagens disponíveis de maneira assíncrona ou sob demanda, dependendo da configuração. Essa flexibilidade permite que o consumidor processe as mensagens no seu próprio

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

ritmo, garantindo que o sistema funcione de forma eficiente mesmo em cenários de alta carga.

Por exemplo, em uma aplicação de e-commerce, um consumidor pode ser responsável por ler mensagens de uma fila de pedidos e, a partir dessas informações, realizar tarefas como atualizar o inventário, gerar uma nota fiscal ou acionar o sistema de envio.

Os consumidores transformam as mensagens armazenadas nas filas em ações concretas, permitindo que o RabbitMQ integre diversos sistemas e garanta a comunicação fluida entre eles.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu os fundamentos do RabbitMQ como um broker de mensagens para sistemas distribuídos. Entendemos sua definição como um intermediário que processa mensagens de forma assíncrona, facilitando a comunicação entre produtores e consumidores.

Discutimos o funcionamento básico da arquitetura do RabbitMQ, desde o envio de mensagens pelos producers para as exchanges e o roteamento para as queues até o consumo pelos consumers, que são as aplicações finais.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

RABBITMQ. AMQP Benchmarks. 2024. Disponível em: https://www.rabbitmq.com/blog/2024/08/21/amqp-benchmarks. Acesso em: 21 jan. 2025.

RABBITMQ. Documentação Oficial do RabbitMQ. 2024. Disponível em: https://www.rabbitmq.com/docs. Acesso em: 21 jan. 2025.

RABBITMQ. Tutoriais do RabbitMQ. 2024. Disponível em: https://www.rabbitmq.com/tutorials. Acesso em: 21 jan. 2025.

FIAP

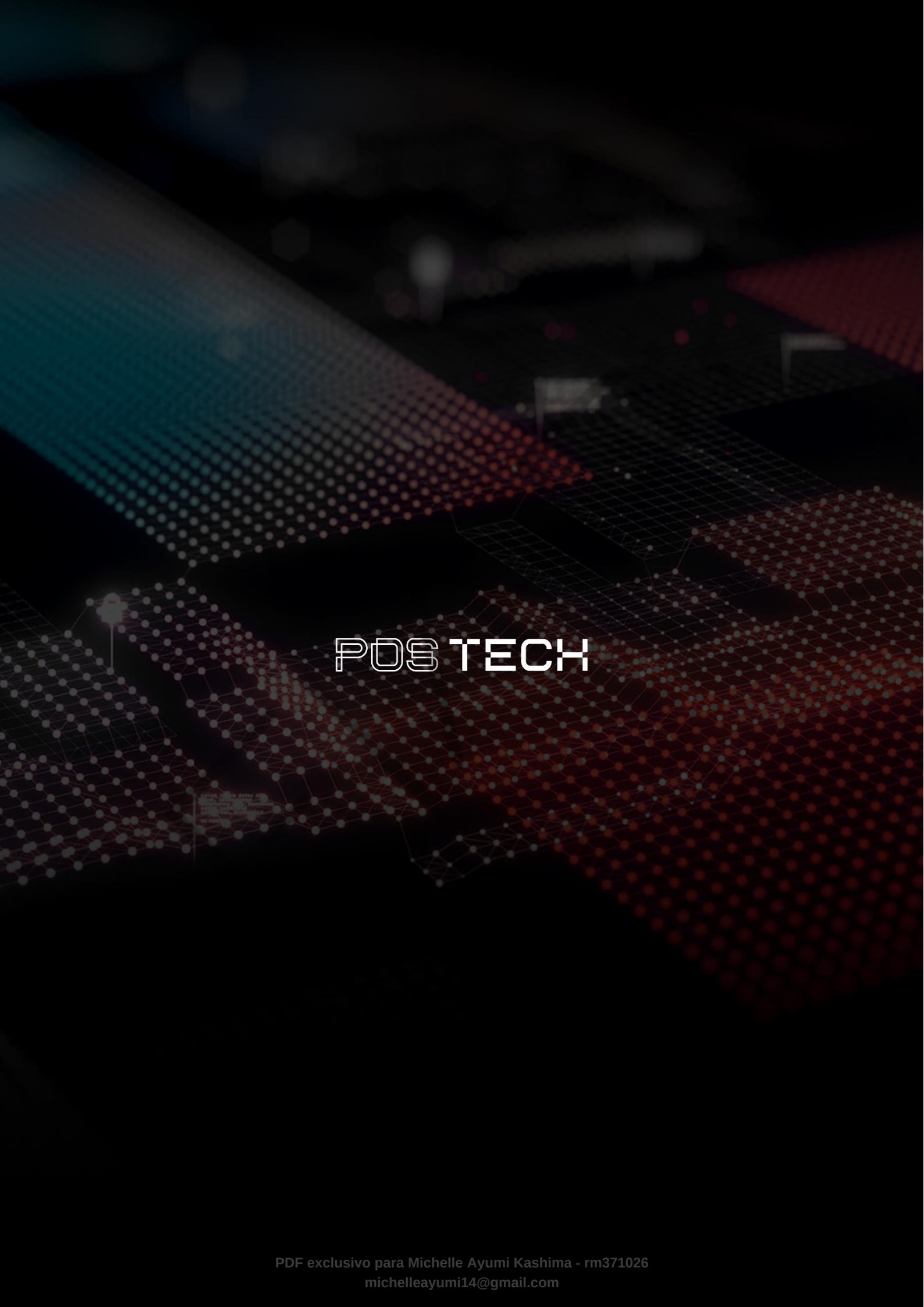
PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Mensageria. Message Broker. Fila de Mensagens. Pub/Sub. Producers. Consumers. Queues. Exchanges. Roteamento de Mensagens. AMQP. RabbitMQ. Apache Kafka. AWS SQS. Escalabilidade. Baixo Acoplamento. Persistência de Mensagens. Garantia de Entrega. Resiliência. Comunicação Assíncrona. Sistemas Distribuídos. Eventos. JSON. XML. Confiabilidade. Cloud-Native. Processamento em Lote. Boas Práticas. Integração de Sistemas. Protocolos de Mensageria. Paginação de Mensagens. Alto Desempenho.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com